VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-2

1. Program to implement Priority scheduling algorithm in preemptive mode.

Aim: : To implement Priority scheduling algorithm in preemptive mode given and finding average Waiting time and average Turnaround time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) of each process.
4. Initialize:
    - Current Time t = 0
    - Completed Processes count = 0
5. Repeat until all processes are completed:
    - Find all processes whose AT ≤ t and are not completed.
    - Select the process with the **highest priority** (smallest priority number).
    - If no process is available, increment the current time by 1.
    - Otherwise:
        - Execute the selected process completely.
        - Calculate:
            - Completion Time (CT) = Current Time + Burst Time
            - Turnaround Time (TAT) = CT − AT
            - Waiting Time (WT) = TAT − BT
        - Mark the process as completed.
        - Update the current time to the Completion Time.
6. Calculate:
    - Average Waiting Time = ΣWT / n
    - Average Turnaround Time = ΣTAT / n
7. Display Process ID, Arrival Time, Burst Time, Priority, Completion Time, Turnaround Time, and Waiting Time.
8. Stop.

Problem:

Calculate Average waiting time , Average turn around time using Priority Scheduling Algorithm in pre-emptive mode for the following processes given below.

Process	AT	BT	Priority
P1	0	4	2
P2	1	3	1
P3	3	4	4
P4	2	2	3
P5	4	1	2

Program:

```
#include <stdio. h>

int main()
{
   int n, i, time = 0, completed = 0;
   int at[20], bt[20], rt[20], pr[20];
   int ct[20], tat[20], wt[20];
   int index, minPriority;
   float avgWT = 0, avgTAT = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time, Burst Time and Priority of P%d: ", i + 1);
      scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
      rt[i] = bt[i];
   }

   while(completed < n)
   {
      index = -1;
      minPriority = 9999;

      for(i = 0; i < n; i++)
      {
         if(at[i] <= time && rt[i] > 0)
```

```
        {
            if(pr[i] < minPriority)
            {
                minPriority = pr[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {
        time++;
    }
    else
    {
        rt[index]--;
        time++;

        if(rt[index] == 0)
        {
            completed++;
            ct[index] = time;
            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];

            avgTAT += tat[index];
            avgWT += wt[index];
        }
    }
}

printf("\n----------------------------------------------------------------\n");
```

```
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    printf("-----------------------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }

    printf("-----------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);
    return 0;
}
```

OUTPUT

TESTCASE 1:

```
71A6225\PRIORITY.exe
Enter number of processes: 5

Enter Arrival Time, Burst Time and Priority for each process:
P1: 0 4 2
P2: 1 3 1
P3: 2 5 4
P4: 3 2 3
P5: 4 1 2

-----------------------------------------------------------
Process AT      BT      Priority        CT      WT      TAT
-----------------------------------------------------------
P1      0       4       2               7       3       7
P2      1       3       1               4       0       3
P3      2       5       4               15      8       13
P4      3       2       3               10      5       7
P5      4       1       2               8       3       4
-----------------------------------------------------------
Average Waiting Time = 3.80
Average Turnaround Time = 6.80

Process returned 0 (0x0)   execution time : 35.929 s
Press any key to continue.
```

TESTCASE 2:

```
71A6225\PRIORITY.exe
Enter number of processes: 3

Enter Arrival Time, Burst Time and Priority for each process:
P1: 0 6 1
P2: 2 5 2
P3: 1 2 0

------------------------------------------------------------
Process AT      BT      Priority        CT      WT      TAT
------------------------------------------------------------
P1      0       6       1               8       2       8
P2      2       5       2               13      6       11
P3      1       2       0               3       0       2
------------------------------------------------------------
Average Waiting Time = 2.67
Average Turnaround Time = 7.00
```

TESTCASE 3:

```
71A6225\PRIORITY.exe
Enter number of processes: 3

Enter Arrival Time, Burst Time and Priority for each process:
P1: 2 3 0
P2: 3 8 3
P3: 1 4 1

------------------------------------------------------------
Process AT      BT      Priority        CT      WT      TAT
------------------------------------------------------------
P1      2       3       0               5       0       3
P2      3       8       3               16      5       13
P3      1       4       1               8       3       7
------------------------------------------------------------
Average Waiting Time = 2.67
Average Turnaround Time = 7.67
```

2. Program to implement Round Robin with Quantum values 2 and 10scheduling algorithm in non-preemptive mode.

Aim: To implement Round Robin scheduling algorithm with Quantum values 2 and 10 given and finding average Waiting time and average Turnaround time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Burst Time (BT) for each process.
4. Read the Time Quantum (TQ).
5. Initialize:
    - Remaining Time (RT) = BT
    - Current Time = 0
6. Repeat until all processes are completed:
    - Traverse all processes.
    - If a process has remaining burst time:
        - If RT > TQ
            - Execute it for TQ units.
            - Reduce remaining time by TQ.
            - Increase current time by TQ.
        - Otherwise
            - Execute it for the remaining time.
            - Increase current time.
            - Calculate:
                - CT = Current Time
                - TAT = CT
                - WT = TAT − BT
7. Calculate:
    - Average Waiting Time
    - Average Turnaround Time

8. Display the results.
9. Stop.

Problem:

Process	AT	BT
P1	4	7
P2	1	5
P3	2	2
P4	3	1

Program:

```
#include <stdio.h>

int main()
{
   int n, tq;
   int at[20], bt[20], rt[20];
   int ct[20], tat[20], wt[20];
   int completed = 0, time = 0;
   int i, found;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]);
      rt[i] = bt[i];
   }

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   while(completed < n)
   {
      found = 0;

      for(i = 0; i < n; i++)
```

```
    {
        if(at[i] <= time && rt[i] > 0)
        {
            found = 1;

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                completed++;
            }
        }
    }

    if(found == 0)
        time++;
}

float avgWT = 0, avgTAT = 0;

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
```

```
 avgWT += wt[i];
 avgTAT += tat[i];
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
     i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
      avgWT /= n;
    avgTAT /= n;
     printf("\nAverage Turnaround Time = %.2f", avgTAT);
    printf("\nAverage Waiting Time = %.2f\n", avgWT);
 return 0;
}
```

OUTPUT

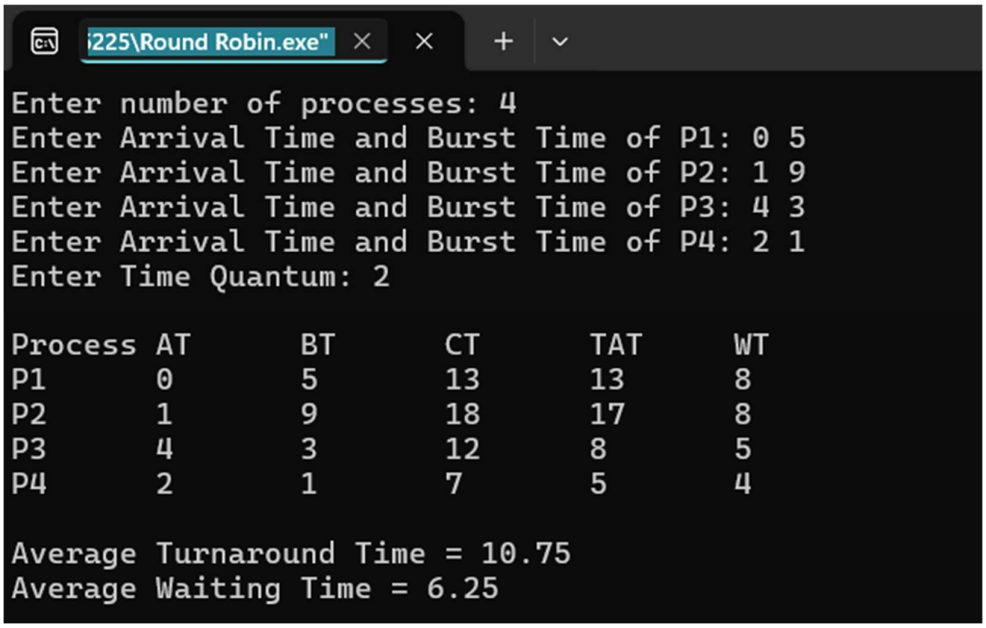
TESTCASE 1:

```
5071A6225\Round.exe"
Enter number of processes: 4
Enter Arrival Time and Burst Time of P1: 4 7
Enter Arrival Time and Burst Time of P2: 1 5
Enter Arrival Time and Burst Time of P3: 2 2
Enter Arrival Time and Burst Time of P4: 3 1
Enter Time Quantum: 2

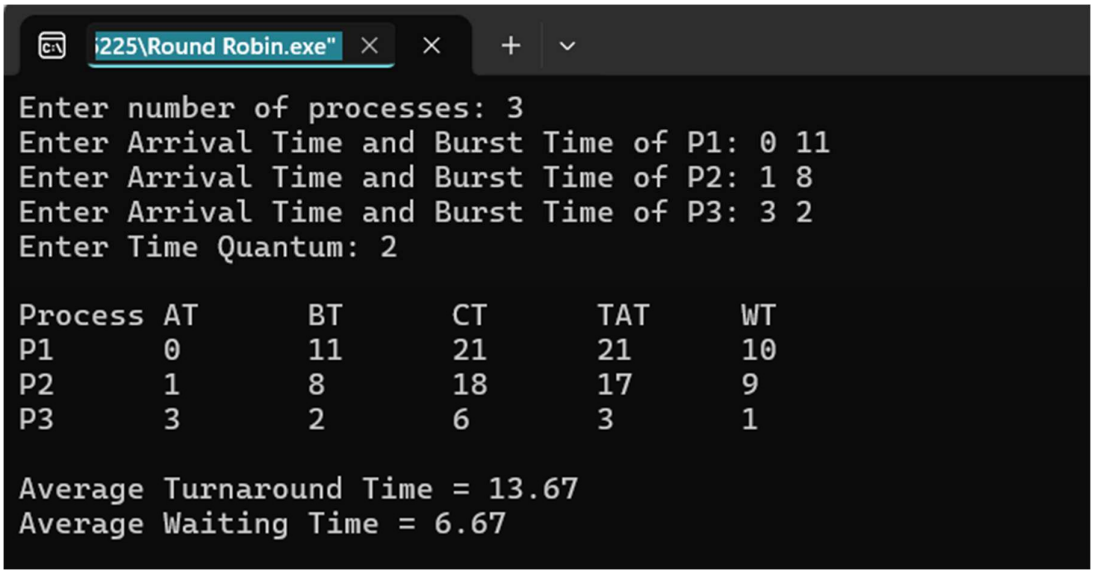
Process AT      BT      CT      TAT     WT
P1      4       7       16      12      5
P2      1       5       13      12      7
P3      2       2       5       3       1
P4      3       1       6       3       2

Average Turnaround Time = 7.50
Average Waiting Time = 3.75
```

TESTCASE 2:



TESTCASE 3:



VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

WITH QUANTUM=10

```
i225\Round Robin.exe"
Enter number of processes: 4
Enter Arrival Time and Burst Time of P1: 4 7
Enter Arrival Time and Burst Time of P2: 1 5
Enter Arrival Time and Burst Time of P3: 2 2
Enter Arrival Time and Burst Time of P4: 3 1
Enter Time Quantum: 10

Process AT      BT      CT      TAT     WT
P1      4       7       16      12      5
P2      1       5       6       5       0
P3      2       2       8       6       4
P4      3       1       9       6       5

Average Turnaround Time = 7.25
Average Waiting Time = 3.50
```

Problem: Calculate the average waiting time and average turn around time for processes given below of all algorithms.

Process	AT	BT
P1	0	8
P2	1	6
P3	2	4
P4	3	2
P5	4	7

Round robin quantum number=2
Priority =5 2 3 1 4

Algorithm	**Average Waiting Time(ms)**	**Average Turnaround Time(ms)**
FCFS(With Arrival Time)	10.0	15.4
FCFS(Without Arrival time)	12.0	17.4
SJF (Non-Preemptive)	8.4	13.8
SRTF (Pre-Emptive)	7.2	12.6
Priority (Pre-Emptive)	7.4	12.8
Round Robin (Quantum = 2)	12.4	17.8

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

